SiCaS

Safe-Rust Side-Channel Attenuation and Leakage Assessment

White Paper

SiCaS Contributors

Version 1.0.0 31 October 2026

Document	White paper
Version	1.0.0
Date	31 October 2026
Code commit	`ef13675`
Release build commit	`123eb69`
Evidence tag	`final-audit-2026-10-31-dpa`
Public label	SAL-2 locally
Production status	Not certified, not audited, not hardware-validated
License	GPL-3.0-only

Abstract

Side-channel leakage is a practical threat to implementations of cryptographic algorithms: power consumption, electromagnetic radiation, and execution time can reveal information about secret keys even when the algorithm is mathematically secure. SiCaS is a safe-Rust, zero-runtime-dependency workspace that combines first-order Boolean-masked primitives and a masked ChaCha20 reference integration with a leakage-assessment harness for TVLA, CPA, DPA, and timing analysis. This white paper explains the problem, the threat model, the architecture, the masking construction, the assessment methodology, the evidence package, and the limitations of the current release. The project is deliberately local-first and evidence-first: every security claim is mapped to a proof obligation, a test, a measurement, or a documented limitation. The current public label is SAL-2 locally; the package is audit-ready but has not been independently audited, red-teamed, or validated on physical hardware. The paper is therefore both a technical overview and a precise statement of what the current evidence does and does not support.

Keywords: side-channel analysis, Boolean masking, TVLA, CPA, DPA, ChaCha20, safe Rust, reproducible builds, assurance.

Contents

1 Executive summary **4**

2 Introduction **4**
2.1 Motivation . . . 4
2.2 Problem statement . . . 5
2.3 Contributions . . . 5
2.4 Non-goals . . . 6

3 Background **6**
3.1 Side-channel analysis . . . 6
3.2 First-order Boolean masking . . . 6
3.3 ChaCha20 and the AES S-box . . . 7
3.4 Safe Rust and zero dependencies . . . 7

4 Threat model **7**
4.1 Assets . . . 7
4.2 Adversary . . . 7
4.3 Assumptions . . . 7
4.4 Out of scope . . . 8

5 System architecture **8**
5.1 Workspace overview . . . 8
5.2 Trust boundaries . . . 8
5.3 Data flow . . . 9

6 Masking and cryptographic design **9**
6.1 Two-share masking . . . 9
6.2 Masked operations . . . 9
6.3 Masked ChaCha20 . . . 10
6.4 Declassification and zeroization . . . 10

7 Leakage-assessment methodology **11**
7.1 Trace format . . . 11
7.2 Preprocessing . . . 11
7.3 TVLA . . . 11
7.4 CPA . . . 11
7.5 DPA . . . 12
7.6 Timing and leakage-model checks . . . 12

8 Security claims and evidence **12**
8.1 Claim taxonomy . . . 12
8.2 Claims C1–C10 . . . 12
8.3 Proof obligations . . . 13
8.4 Assurance levels . . . 13

9 Implementation and release engineering **13**
9.1 Crate structure and API design . . . 13
9.2 Fail-closed entropy . . . 14
9.3 Constant-time discipline . . . 14
9.4 Release and supply chain . . . 14

10 Evaluation **14**
10.1 Functional correctness . . . 14
10.2 Memory safety and undefined behaviour . . . 15
10.3 Model checking . . . 15
10.4 Mutation, coverage, and fuzzing . . . 15
10.5 Cross-architecture and performance . . . 15
10.6 Summary . . . 15

11 Limitations and open gaps **16**
11.1 No hardware leakage evidence . . . 16
11.2 No independent audit or red team . . . 16
11.3 Higher-order, fault, and invasive attacks . . . 16
11.4 Template attacks . . . 16
11.5 Performance gap . . . 16
11.6 Zeroization . . . 16
11.7 Platform coverage . . . 16

12 Roadmap to SAL-4 **16**

13 Related work **17**

14 Conclusion **17**

A Glossary **17**

B Claim-to-evidence matrix **18**

C Reproducibility commands **18**

D Document history **18**

1 Executive summary

SiCaS addresses two problems that are usually handled by separate teams and separate toolchains:

1. **Attenuation.** How can cryptographic software be structured so that first-order power or electromagnetic leakage is reduced, and how can the masking construction be made explicit and reviewable?
2. **Assessment.** How can raw power or electromagnetic traces be analysed reproducibly, with a clear leakage model, published parameters, and evidence that can be independently checked?

The project keeps both sides in one workspace. The `sicas-core` crate provides constant-time primitives, an opaque `Secret<T>` wrapper, two-share Boolean masking, and a masked adder. The `sicas-reference` crate exercises those gadgets in a first-order masked ChaCha20 implementation, including a 64-block bitsliced batch path. The `sicas-harness` crate provides a strict trace format, preprocessing, TVLA, CPA, AES S-box DPA, dudect-style timing checks, and synthetic validation. The `sicas-platform` crate supplies a fail-closed OS entropy adapter. The `sicas-zeroize` crate is an isolated, non-published, unaudited experiment in volatile zeroization.

The main contributions of the current release are:

- a safe-Rust, zero-runtime-dependency implementation of the core masking and constant-time building blocks;
- a masked ChaCha20 reference integration with RFC 8439 validation and a differential test against an unmasked reference;
- a 64-block bitsliced batch path that meets the project's 10× hard performance ceiling;
- a leakage-assessment harness covering TVLA, CPA, DPA, timing, preprocessing, and synthetic validation;
- a formalised claim and proof-obligation structure, a reproducible evidence bundle, and a local proof runner;
- a precise assurance statement: **SAL-2 locally**, not SAL-3 or SAL-4.

The paper is organised as follows. Section 2 introduces the problem and contributions. Section 3 reviews side-channel analysis and masking. Section 4 states the threat model. Section 5 describes the architecture. Section 6 explains the masking and cryptographic design. Section 7 describes the leakage-assessment methodology. Section 8 maps claims to evidence. Section 9 discusses implementation and release engineering. Section 10 summarises the evaluation. Section 11 states the limitations. Section 12 gives the roadmap to SAL-4. Section 13 discusses related work, and Section 14 concludes. Appendices provide a glossary, a claim-to-evidence matrix, reproducibility commands, and the document history.

2 Introduction

2.1 Motivation

A cryptographic implementation can be mathematically secure and still leak its key through its physical behaviour. Kocher's timing attack (Kocher, 1996) showed that data-dependent execution time can reveal secret information. Differential power analysis (Kocher et al., 1999) and correlation power analysis (Brier et al., 2004) extended the idea to power consumption and electromagnetic radiation. Template attacks (Chari et al., 2002) showed that a profiled

adversary can recover keys from a small number of traces. The practical lesson is that security must be argued at the level of the implementation, not only at the level of the algorithm.

Masking is one of the standard countermeasures. A secret value is split into shares, and every operation is performed on the shares so that no single share (and, for first-order masking, no single leakage point) depends on the secret. The theoretical foundations go back to private circuits (Ishai et al., 2003) and threshold implementations (Nikova et al., 2006); practical masking schemes for AES and other algorithms are surveyed by Mangard et al. (2007) and Rivain and Prouff (2010). Masking is not a proof of security by itself. It depends on fresh randomness, a correct gadget for every operation, a leakage model that matches the device, and an evaluation methodology that can detect implementation mistakes.

2.2 Problem statement

The project addresses the following engineering problem:

> Build a small, auditable, safe-Rust workspace that (i) provides first-order Boolean-masked primitives and a reference integration, (ii) provides a reproducible leakage-assessment harness, and (iii) records every security claim with explicit evidence and limitations, without third-party runtime dependencies and without overstating the assurance level.

The problem has three constraints that shape the design:

1. **Safety and auditability.** The implementation should use safe Rust and avoid unnecessary abstraction. The published surface should be small enough to review.
2. **Supply-chain minimalism.** The workspace should have zero third-party runtime dependencies. Tooling dependencies are allowed only outside the published crates and are documented.
3. **Evidence discipline.** A claim that cannot be reproduced from the repository is not a claim. Missing evidence downgrades the claim.

2.3 Contributions

The release described in this paper makes the following contributions:

1. **Core primitives.** A `no_std`, `forbid(unsafe_code)` crate with constant-time comparison, selection, swap, and table lookup; an opaque `Secret<T>` type; and a two-share `Masked<T>` type with masked XOR, AND, shifts, rotations, selection, equality, bit extraction, refresh, and addition.
2. **Reference integration.** A first-order masked ChaCha20 implementation with RFC 8439 validation, differential testing against an unmasked reference, per-block refresh, and a 64-block bitsliced batch path.
3. **Assessment harness.** A strict binary trace format, preprocessing, fixed-vs-random TVLA, byte-oriented CPA with custom leakage models, AES S-box DPA, dudect-style timing checks, synthetic trace generation, and a first-order leakage-model check with a negative control.
4. **Evidence system.** A claim matrix, proof obligations, a local proof runner, mutation, coverage, fuzzing, and cross-architecture evidence, a reproducible release manifest, an SBOM, provenance metadata, a signing dry run, and a hashed evidence bundle.
5. **Assurance statement.** A precise local label (SAL-2) and an explicit list of external gates (independent audit, red team, hardware leakage measurement, production signing) that are not simulated.

2.4 Non-goals

The project does not aim to be a general-purpose cryptographic library, a compiler pass for automatic masking, a hardware root of trust, or a certified product. It does not claim zero leakage, higher-order security, fault resistance, invasive-attack resistance, cache-timing resistance, or production zeroization. It does not claim SAL-4. These non-goals are part of the design: they keep the claims small enough to verify.

3 Background

3.1 Side-channel analysis

A side-channel adversary observes a physical quantity that depends on the computation. Common channels are execution time, power consumption, electromagnetic radiation, and cache behaviour. The adversary typically knows or chooses the plaintext and records traces $T_i[j]$, where i indexes the trace and j the sample.

Three analysis families are relevant to this project:

TVLA. Test Vector Leakage Assessment compares a fixed-input group with a random-input group using Welch's t-test at each sample (Goodwill et al., 2011; Schneider and Moradi, 2015). A common threshold is $|t| > 4.5$.

CPA. Correlation power analysis correlates a predicted leakage model, such as the Hamming weight of an intermediate value, with the observed traces (Brier et al., 2004).

DPA. Differential power analysis partitions traces by a predicted bit and compares the means of the partitions (Kocher et al., 1999). A nonlinear intermediate such as an S-box output is required for key-dependent partitions.

Timing analysis is a separate channel. Dudect-style tests compare fixed and random classes and use Welch's t-test (Reparaz et al., 2017). Template attacks (Chari et al., 2002) are a profiled attack and are outside the current threat model.

3.2 First-order Boolean masking

Let $x \in \{0,1\}^n$. A two-share Boolean sharing is

$$\mathsf{Share}(x) = (x \oplus r,\ r),$$

where $r \in \{0,1\}^n$ is uniform and independent of x. Reconstruction is

$$\mathsf{Rec}(s_0, s_1) = s_0 \oplus s_1.$$

Each share is individually uniform: $s_1 = r$ is uniform, and $s_0 = x \oplus r$ is uniform for every fixed x. Therefore a single-share leakage function that depends only on one share reveals no information about x in the ideal model.

Linear operations are easy: XOR is share-wise, and complement flips one share. AND and addition require fresh randomness. The masked AND used in this project is

$$c_0 = (a_0 \wedge b_0) \oplus r, \tag{1}$$
$$c_1 = (a_0 \wedge b_1) \oplus (a_1 \wedge b_0) \oplus (a_1 \wedge b_1) \oplus r, \tag{2}$$

with fresh uniform r. The XOR of the shares is $(a_0 \oplus a_1) \wedge (b_0 \oplus b_1) = a \wedge b$.

Masked addition is a ripple-carry adder over masked bits. At bit i, the sum bit is $a_i \oplus b_i \oplus c_i$, and the carry is $(a_i \wedge b_i) \oplus (a_i \wedge c_i) \oplus (b_i \wedge c_i)$, shifted to the next position. Each AND receives a fresh mask.

3.3 ChaCha20 and the AES S-box

ChaCha20 is an ARX stream cipher (Bernstein, 2008; Nir and Langley, 2018). Its state is sixteen 32-bit words: four constants, eight key words, a counter, and three nonce words. The quarter round uses 32-bit addition, XOR, and rotations by 16, 12, 8, and 7. Ten double rounds produce the keystream block after a final addition of the initial state. The ARX structure is attractive for masking because addition, XOR, and rotation have relatively small masked gadgets.

The AES S-box (National Institute of Standards and Technology, 2001) is used in the DPA harness as a nonlinear target. A single-bit partition of the S-box output $\mathsf{SBOX}[p \oplus k]$ is key-dependent, which makes key recovery possible on suitable traces. This is a deliberate harness feature, not a claim that the reference ChaCha20 implementation uses AES.

3.4 Safe Rust and zero dependencies

Rust's ownership and borrowing rules prevent many memory-safety defects, and the `forbid(unsafe_code)` lint makes the absence of unsafe code a mechanical property of the core, reference, platform, and harness crates. RustBelt (Jung et al., 2018) provides a formal foundation for Rust's safety claims. The project still treats compiler behaviour as an assumption: constant-time properties are checked by assembly inspection and timing tests, not assumed from the language.

Zero runtime dependencies reduce the supply-chain surface. The workspace uses only the Rust standard library and core. The non-published fuzz crate uses `libfuzzer-sys` for fuzzing and is excluded from the workspace.

4 Threat model

4.1 Assets

The assets are long-term and session cryptographic keys, plaintext and intermediate secret values, mask shares and entropy-source state, and control-flow or memory-access patterns derived from secrets.

4.2 Adversary

The v1.0 adversary is a passive first-order side-channel adversary with:

- a power or electromagnetic probe;
- an oscilloscope and a trigger signal;
- known or selected plaintext;
- a bounded number of traces from the same device under normal operation.

The trace budget is up to 10^6 traces for first-order TVLA/DPA and up to 10^5 traces for CPA. Exact per-target numbers must be published with results.

4.3 Assumptions

1. The OS CSPRNG or injected entropy source is secure when it reports success.
2. The compiler and toolchain are not malicious.
3. The target device is not physically modified.
4. The attacker cannot observe the device during a trusted setup phase.

5. The leakage model used in an assessment is appropriate for the target; a passing test under a wrong model is not evidence of absence of leakage.

4.4 Out of scope

Higher-order DPA, profiling template attacks, fault injection, invasive attacks, cache-timing and microarchitectural attacks, supply-chain attacks, side channels in the OS or I/O, malicious hardware, and production zeroization are outside the claimed threat model. They may be discussed as hardening evidence, but they are not security claims.

5 System architecture

5.1 Workspace overview

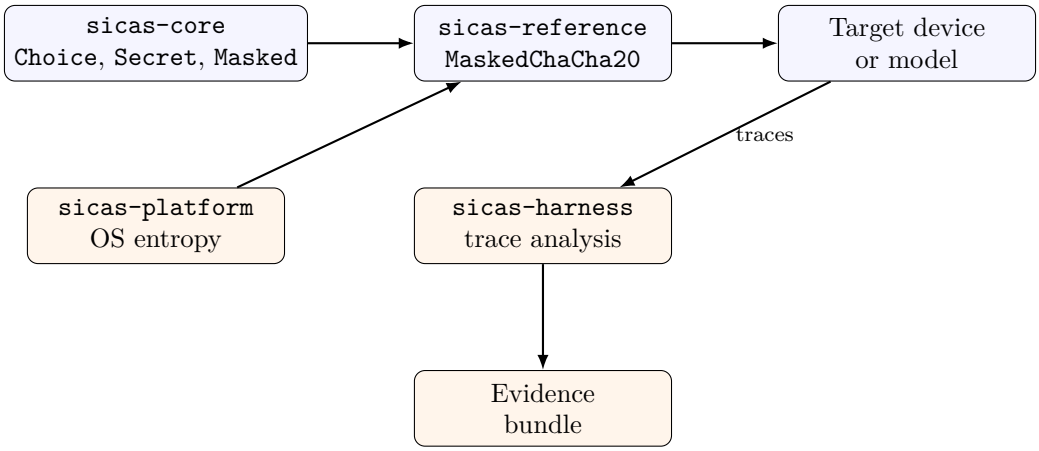


Figure 1: SiCaS separates runtime building blocks from offline leakage assessment. The reference integration is a validation target; the harness consumes traces and produces evidence.

The workspace contains five crates and a non-published fuzz crate:

`sicas-core` Constant-time primitives, `Secret<T>`, two-share `Masked<T>`, masked addition, and the fail-closed `EntropySource` trait. `no_std`, `forbid(unsafe_code)`, zero runtime dependencies.

`sicas-reference` First-order masked ChaCha20. `no_std`, `forbid(unsafe_code)`, zero runtime dependencies.

`sicas-harness` Trace format, preprocessing, TVLA, CPA, DPA, timing, synthetic generation, leakage-model checks, and CLI. `std`, `forbid(unsafe_code)`, zero runtime dependencies.

`sicas-platform` Safe OS entropy adapter. `std`, `forbid(unsafe_code)`, zero runtime dependencies. Unsupported platforms fail closed.

`sicas-zeroize` Experimental volatile zeroization boundary. `no_std`, isolated unsafe, non-published, unaudited, and not used by the published crates.

5.2 Trust boundaries

There are four explicit boundaries:

1. **Entropy boundary.** The caller supplies an `EntropySource`. A failure must propagate as an error; there is no weak fallback.

2. **Declassification boundary.** The declassification points are `Secret::expose_secret`, `Secret::into_inner`,`Masked::unmask`,and the ChaCha20 keystream output. They expose plaintext values. They are documented and must be justified by the caller's leakage contract.
3. **Device boundary.** The harness consumes traces from an external acquisition setup. It does not control the probe, the oscilloscope, or the target.
4. **Release boundary.** The release manifest, SBOM, provenance, and signing dry run are separate from the runtime code. Production signing is not claimed.

5.3 Data flow

A typical assessment is:

1. acquire traces and plaintexts;
2. serialise traces in the SiCaS binary format;
3. trim, filter, and align;
4. run TVLA, CPA, or DPA;
5. publish the raw traces, parameters, tool versions, commit, and result.

The harness never treats a passing test as a proof of absence of leakage. A result is evidence for the tested target, trace count, leakage model, and acquisition setup.

6 Masking and cryptographic design

6.1 Two-share masking

Definition 6.1 (Two-share Boolean sharing)**.** For $x \in \{0,1\}^n$, a two-share Boolean sharing is a pair $(s_0, s_1) \in \{0,1\}^n \times \{0,1\}^n$ such that $s_0 \oplus s_1 = x$. The sharing is *fresh* if the mask used to construct it is uniform and independent of x and of all other masks.

Theorem 6.2 (Single-share uniformity)**.** *Let r be uniform on $\{0,1\}^n$ and independent of x. Then $s_1 = r$ and $s_0 = x \oplus r$ are each uniform on $\{0,1\}^n$, and the marginal distribution of either share is independent of x.*

Proof. For every $y \in \{0,1\}^n$, $\Pr[s_1 = y] = 2^{-n}$. For s_0, $\Pr[s_0 = y] = \Pr[r = y \oplus x] = 2^{-n}$. The marginals do not depend on x. □

This is the ideal first-order argument. It does not model glitches, transitions, register reuse, or physical leakage that depends on both shares. Those effects are outside the current claim and are why hardware validation remains necessary.

6.2 Masked operations

Theorem 6.3 (Masked AND correctness)**.** *For all $a, b \in \{0,1\}^n$ and all shares a_0, a_1, b_0, b_1 with $a_0 \oplus a_1 = a$ and $b_0 \oplus b_1 = b$, the shares c_0, c_1 in Table 1 satisfy $c_0 \oplus c_1 = a \wedge b$.*

Proof. Expand the four products:

$$(a_0 \oplus a_1) \wedge (b_0 \oplus b_1) = (a_0 \wedge b_0) \oplus (a_0 \wedge b_1) \oplus (a_1 \wedge b_0) \oplus (a_1 \wedge b_1).$$

The two occurrences of r cancel under XOR. □

Table 1: Masked operations and their correctness identities.

Operation	Identity / implementation
XOR	$(a_0 \oplus b_0,\ a_1 \oplus b_1)$
Complement	$(\neg a_0,\ a_1)$
Refresh	$(a_0 \oplus r,\ a_1 \oplus r)$ with fresh r
AND	$c_0 = (a_0 \wedge b_0) \oplus r$, $c_1 = (a_0 \wedge b_1) \oplus (a_1 \wedge b_0) \oplus (a_1 \wedge b_1) \oplus r$
Addition	ripple-carry over masked bits with a fresh mask per AND
Select	select each share independently with a public `Choice`
Equality	compare $a_0 \oplus b_0$ and $a_1 \oplus b_1$ in constant time

Theorem 6.4 (Masked addition correctness)**.** *For all $a, b \in \{0,1\}^{32}$, the ripple-carry masked adder reconstructs $(a + b) \bmod 2^{32}$.*

Proof sketch. At bit i, the reconstructed sum bit is $a_i \oplus b_i \oplus c_i$, and the reconstructed carry is $(a_i \wedge b_i) \oplus (a_i \wedge c_i) \oplus (b_i \wedge c_i)$. This is the standard full-adder identity. Induction on i from the zero carry gives the 32-bit sum modulo 2^{32}. The masked AND gadgets preserve the reconstructed values by Theorem 6.3, so the masked implementation computes the same function. □

The project's Kani harnesses check the core masked operations for all inputs of the specified widths, including addition, refresh, selection, equality, and share reconstruction. These are model-checking results for the Rust implementation, not a mechanized proof of physical security.

6.3 Masked ChaCha20

The scalar masked ChaCha20 state is an array of sixteen `Masked<u32>` words. The initial state is masked with fresh randomness. For each block, the working state and the initial state used by the final addition are independently refreshed. This avoids feeding correlated shares into the masked adder. The quarter round uses masked addition, XOR, and rotation; the final addition uses the masked adder; the output is unmasked only at the keystream boundary.

The bitsliced batch path processes 64 blocks at once. Each bit position of each 32-bit word is represented by a `Masked<u64>` plane: the 64 bits of the plane correspond to the 64 lanes. The quarter round is performed on planes; the masked adder is a ripple-carry adder over the 32 bit planes. After the final addition, the planes are transposed into 64 lane-major 64-byte blocks. The transpose is a 64×64 bit-matrix transpose implemented as a butterfly network with widths 32, 16, 8, 4, 2, and 1.

The bitsliced path is a performance optimisation, not a different security claim. It uses the same two-share masking and fresh-mask discipline. Its output is validated against the scalar and unmasked references, including counter wrap and batch/scalar interoperability.

6.4 Declassification and zeroization

The following operations are declassification boundaries:

- `Secret::expose_secret` and `Secret::into_inner`;
- `Masked::unmask`;
- the ChaCha20 keystream output;
- the bitsliced transpose, which unmasks the output planes before writing bytes.

The core and reference types do not guarantee zeroization on drop. The `sicas-zeroize` crate is an experimental volatile-write boundary with a compiler fence and a drop-path test. It is non-published, unaudited, and not integrated into the published crates. A production zeroization claim would require an audited platform adapter and a review of register, temporary, and move semantics.

7 Leakage-assessment methodology

7.1 Trace format

The trace format is deliberately small and strict:

```
magic:              8 bytes, b"SiCaSTR1"
version:            u16 little-endian, currently 1
samples_per_trace:  u32 little-endian
trace_count:        u32 little-endian
samples:            trace_count * samples_per_trace i16 little-endian
```

The limits are 10^6 traces, 10^6 samples per trace, and 10^8 total samples. The parser rejects invalid magic, unsupported versions, zero dimensions, ragged traces, oversized inputs, and truncated files with typed errors. It is exercised by a libFuzzer target and by Miri.

7.2 Preprocessing

Trim. Keep the half-open sample range $[s, e)$.

Filter. Apply a centered moving average with a window of w samples and clipped edges. A window of one is the identity.

Align. Choose the shift in $[-m, m]$ that maximises the normalized cross-correlation with a reference trace, then shift and zero-fill.

Preprocessing is part of the evidence. The parameters and the reference trace must be published with the result.

7.3 TVLA

For each sample j, let $x_{0,j}$ and $x_{1,j}$ be the fixed and random groups. Welch's statistic is

$$t_j = \frac{\bar{x}_{0,j} - \bar{x}_{1,j}}{\sqrt{s_{0,j}^2/n_0 + s_{1,j}^2/n_1}},$$

where s^2 is the sample variance. The implementation computes the statistic from sums and sums of squares, clamps negative variances to zero, and reports the maximum $|t|$, its sample index, and the number of samples above the threshold. The default threshold is 4.5.

TVLA is a detection test, not a proof of absence of leakage. A pass means that the test did not detect a difference at the chosen threshold, trace count, and model.

7.4 CPA

For a key guess k, plaintext p_i, sample j, and leakage model $M(k, p_i)$, CPA computes the Pearson correlation

$$\rho_{k,j} = \frac{\sum_i (M(k,p_i) - \bar{M}_k)(T_{i,j} - \bar{T}_j)}{\sqrt{\sum_i (M(k,p_i) - \bar{M}_k)^2 \sum_i (T_{i,j} - \bar{T}_j)^2}}.$$

The best key is the one with the largest absolute correlation across samples. The default model is $\mathsf{HW}(p_i \oplus k)$. The Hamming-weight model is invariant under key complement: $\mathsf{HW}(x \oplus k)$ and $\mathsf{HW}(x \oplus k \oplus 0xFF)$ are perfectly negatively correlated, so the tie-break returns the lower key. The library exposes `cpa_with_model` for algorithm-specific models such as an AES S-box.

7.5 DPA

For each key guess k, bit b, and sample j, partition the traces by

$$\mathsf{SBOX}[p_i \oplus k]_b.$$

Let S_0 and S_1 be the two partitions. The difference of means is

$$\Delta_{k,b,j} = \frac{1}{|S_1|} \sum_{i \in S_1} T_{i,j} - \frac{1}{|S_0|} \sum_{i \in S_0} T_{i,j}.$$

The report records the key, bit, sample, absolute difference, and signed difference of the largest absolute difference. The search uses a strict $>$ comparison, so the first maximum wins.

The AES S-box supplies the nonlinearity that makes the partition key-dependent. A single-bit partition may not uniquely recover the full key from noisy traces; several bits or samples should be combined before treating a key as recovered. The synthetic tests recover the exact key from noise-free single-bit leaks at bit 0 and bit 3, including the sign of the difference.

7.6 Timing and leakage-model checks

The `timing` and `ct` commands run dudect-style fixed-vs-random tests (Reparaz et al., 2017) and report a t-statistic. The `leakage-check` command measures the Pearson correlation between individual shares and the secret for a masked byte, a masked AND composition, and a masked 32-bit addition. A negative control correlates the secret with itself and must be detected. The command fails if the control is missed or if any masked check exceeds the threshold.

These checks validate the masking model and the analysis code. They are not hardware evidence and do not replace TVLA/CPA/DPA on a physical target.

8 Security claims and evidence

8.1 Claim taxonomy

A claim is a statement about a property, a target, and a scope. The project uses three categories:

1. **Locally proven.** The claim has a test, proof, measurement, or reproducible evidence artifact in the repository.
2. **Documented limitation.** The claim is not made; the limitation is recorded in the gap report.
3. **External gate.** The claim requires hardware, credentials, or an independent party and is not simulated.

8.2 Claims C1–C10

Table 2: Claim matrix (abridged).

ID	Claim	Current local status
C1	The safe API prevents accidental secret-dependent branching/indexing.	Compile-fail tests pass.
C2	Constant-time gadgets are data-independent on supported targets.	Dudect and assembly evidence pass on x86_64; other targets pending.
C3	First-order Boolean masking reduces first-order DPA below the TVLA threshold for the reference algorithm.	Synthetic and model evidence pass; hardware TVLA pending.
C4	Entropy failure is fail-closed.	Local fault tests pass; unsupported platforms return an error.
C5	Zero runtime dependencies.	Verified locally.
C6	No unsafe code in core logic crates.	`forbid(unsafe_code)` verified locally.
C7	Builds are reproducible.	Verified locally across independent target directories.
C8	Critical invariants are model-checked.	30 Kani harnesses pass.
C9	v1.0 meets SAL-4.	Pending independent verification.
C10	Release artifacts are signed and traceable.	Local signing dry run only; production signing pending.

8.3 Proof obligations

The proof obligations are maintained in `docs/PROOF_OBLIGATIONS.md`. They cover memory safety, functional correctness, constant-time behaviour, masking, entropy, parser robustness, statistics, coverage, mutation, supply-chain integrity, release integrity, and performance. Each obligation records the method, the evidence, and the current status. A claim without an obligation is not part of the public claim set.

8.4 Assurance levels

Table 3: SiCaS assurance levels.

Level	Requirements	Allowed public claim
SAL-0	Demo or prototype.	"experimental"
SAL-1	Unit/integration tests and CI.	"tested"
SAL-2	SAL-1 plus Miri, sanitizers, fuzzing, static analysis.	"hardened"
SAL-3	SAL-2 plus formal/model checking, real-hardware TVLA, external audit.	"audited"
SAL-4	SAL-3 plus independent red team, reproducible/signed builds, no open critical/high findings, public evidence bundle.	"SAL-4 under threat model T"

The current label is **SAL-2 locally**. The project does not claim SAL-3 or SAL-4.

9 Implementation and release engineering

9.1 Crate structure and API design

The workspace uses Rust 2024 edition and MSRV 1.99. The shared workspace lints deny `unsafe_code`, missing docs, missing debug implementations, unreachable public items, unused `must_use`, and a set of Clippy pedantic and nursery lints. The release profile uses `opt-level="z"`, fat LTO, one codegen unit, `panic="abort"`, and stripped symbols.

The core API is intentionally small. `Secret<T>` does not implement `PartialEq`, `Ord`, `Hash`, `Index`, `Clone`, or `Copy`. This makes accidental equality, ordering, indexing, and copying a compile error. `Masked<T>` exposes explicit share access only through documented methods. `EntropySource` returns `Result`; there is no weak fallback.

9.2 Fail-closed entropy

The `sicas-platform` adapter reads `/dev/urandom` on Unix-like systems. If the source cannot be opened, it returns `EntropyError::Unavailable`. Other platforms return the same error until a dedicated adapter is added. The core `EntropySource` trait requires implementations to fill the destination completely or return an error; partial fill followed by success is a contract violation.

9.3 Constant-time discipline

The constant-time claim is supported by three layers:

1. branchless formulas in the core gadgets;
2. assembly inspection across optimization levels, codegen units, panic strategies, and LTO variants;
3. dudect-style timing tests on x86_64.

The assembly checker covers 69 configurations: opt-level 0 on four targets, opt-levels 0/1/2/3/s and codegen/panic variants on x86_64 and AArch64, and thin/fat LTO on x86_64. The dudect evidence includes 5×10^6 debug-profile samples and 3×10^6 release-profile samples per check. These are local measurements; they do not cover every compiler, CPU, or physical target.

9.4 Release and supply chain

The release manifest script builds the workspace, copies `sicas-harness`, and writes a SHA-256 checksum, a CycloneDX SBOM (CycloneDX Project, 2023), and provenance metadata. The manifest validator checks the artifact hash, checksum file, provenance, SBOM component graph, workspace versions, timestamp, and (optionally) the current commit. The local signing dry run produces and verifies a detached signature with an ephemeral test key. It is a process check, not a production identity.

The release binary for the current package is 386,232 bytes with SHA-256:

`1f0d304ca9b9582fd9153e7c006c240bd0e06acc3a685dfad93088429e4de7da`

The release build commit is `123eb69`; the proof code commit is `ef13675`. The binary is reproducible across independent target directories.

10 Evaluation

10.1 Functional correctness

The workspace passes its unit, integration, property, and doctest suites. The core tests cover constant-time comparison, selection, swap, lookup, secret handling, masking, refresh, share reconstruction, and masked addition. The reference tests cover RFC 8439 vectors, differential checks against an unmasked reference, lane-independent bitsliced addition, key-plane balance, batch/scalar interoperability, counter wrap, and entropy-failure propagation. The harness tests cover trace parsing, preprocessing, TVLA, CPA, DPA, timing validation, synthetic generation, and leakage-model checks.

10.2 Memory safety and undefined behaviour

Miri (Rust Project, 2024) is run over the workspace, with reduced iteration counts and documented skips for native-only statistical, OS, and subprocess paths. AddressSanitizer (Serebryany et al., 2012) runs over the workspace library tests. Valgrind (Nethercote and Seward, 2007) runs over five library test binaries with `-leak-check=full` and `-errors-for-leak-kinds=definite`. All three report no errors on the covered paths.

10.3 Model checking

Kani (Kani Rust Verifier Project, 2024) is used for 30 harnesses covering constant-time gadgets, masked linear and logical operations, addition, refresh, selection, equality, share reconstruction, and share uniformity. The proofs are bounded to the declared widths and assumptions; they are not a proof of physical security.

10.4 Mutation, coverage, and fuzzing

The current workspace mutation run tests 630 mutants: 574 are caught, 0 are missed, and 56 are unviable. The DPA file is also mutated in a targeted run: 30 mutants, 29 caught, 0 missed, 1 unviable. Line coverage is 97.28% and region coverage is 94.89%; the DPA module is at 100% line coverage. Five libFuzzer campaigns cover masked operations, constant-time lookup, trace parsing, ChaCha20, and the bitsliced batch path at one million executions each.

10.5 Cross-architecture and performance

AArch64 Linux under QEMU passes 141 workspace library tests; i686 native passes 154 all-target tests. The bitsliced masked ChaCha20 batch measures 7.15× versus the unmasked reference in the final full proof, meeting the 10× hard ceiling. The 5× target remains open; reaching it requires SIMD or a faster audited mask stream, both of which would create a new security boundary.

10.6 Summary

11 Limitations and open gaps

11.1 No hardware leakage evidence

The project has no physical power or electromagnetic traces. The synthetic and model-based evidence validates the analysis path and the masking construction, but it does not prove that a device leaks less. Hardware TVLA/CPA/DPA remains the central external gate.

Table 4: Local evidence summary.

Gate	Result
Full local proof	PASS at `8c41e4b`
AES S-box DPA quick proof	PASS at `ef13675`
ASan	141 workspace library tests pass
Kani	30 harnesses pass
Mutation	630 mutants, 574 caught, 0 missed, 56 unviable
Coverage	97.28% lines, 94.89% regions
Fuzzing	Five 1M-execution campaigns
AArch64 QEMU / i686	141 / 154 tests pass
Valgrind	Five binaries, zero errors
Release	386,232 bytes, reproducible, validator and signing dry run pass
Evidence bundle	109 files, hashes verified

11.2 No independent audit or red team

The local self-audit and adversarial review are useful, but they are not independent. An external audit and an independent red team are required before SAL-3/SAL-4.

11.3 Higher-order, fault, and invasive attacks

The masking is first-order. It does not claim security against higher-order DPA, fault injection, invasive probing, cache-timing, or microarchitectural attacks.

11.4 Template attacks

Profiling template attacks are outside the threat model. The harness includes DPA, but not a template-attack implementation. This is a scope decision, not a claim that template attacks are infeasible.

11.5 Performance gap

The 10× hard ceiling is met; the 5× target is open. The remaining gap is in the bitsliced masked adder and entropy stream. SIMD or a faster audited mask stream would require a new proof.

11.6 Zeroization

The experimental zeroization adapter is not a production guarantee. It is non-published, unaudited, and not integrated into the published crates. A production claim requires an audited boundary and a review of register, temporary, and move semantics.

11.7 Platform coverage

The core and reference crates are `no_std` and compile for embedded targets, but runtime evidence is limited to x86_64, AArch64 QEMU, and i686. ARM/RISC-V runtime timing and hardware leakage remain pending. Windows, macOS, iOS, Android, eBPF, and kernel modules are not supported in v1.0.

12 Roadmap to SAL-4

The roadmap separates local work from external gates.

1. **Freeze the local proof.** The current package is the audit-ready local artifact. Any code change requires a new proof run and a new evidence bundle.
2. **Independent audit.** An external auditor reviews the threat model, claims, code, tests, and evidence, and reports findings.
3. **Independent red team.** A red team attempts to break the claims with implementation, statistical, and physical attacks.
4. **Hardware validation.** Acquire power/EM traces from physical targets and run TVLA/CPA/DPA with published parameters and raw data.
5. **Production signing.** Replace the ephemeral signing dry run with a production identity and a real Sigstore/SLSA release run.
6. **Remediation and retest.** Fix findings, rerun the gates, and publish the updated evidence.
7. **SAL-4 decision.** Claim SAL-4 only when every criterion is met and independently verified.

Optional local work includes SIMD for the 5× target, template-attack tooling, Kani coverage for the bitsliced adder, and a reviewed zeroization adapter. These are enhancements, not substitutes for the external gates.

13 Related work

Timing attacks (Kocher, 1996), DPA (Kocher et al., 1999), CPA (Brier et al., 2004), and template attacks (Chari et al., 2002) established the main side-channel analysis families. Mangard et al. (2007) provide a comprehensive treatment of power analysis and countermeasures. Private circuits (Ishai et al., 2003), threshold implementations (Nikova et al., 2006), and probing-secure masking (Rivain and Prouff, 2010) provide the theoretical background for masking. TVLA (Goodwill et al., 2011; Schneider and Moradi, 2015) and ISO/IEC 17825 (ISO/IEC, 2016) provide evaluation methodologies. Dudect (Reparaz et al., 2017) provides a practical constant-time test.

On the implementation side, RustBelt (Jung et al., 2018) provides a formal foundation for Rust safety, Kani (Kani Rust Verifier Project, 2024) provides bounded model checking, libFuzzer (LLVM Project, 2024) provides coverage-guided fuzzing, and mutation testing (Jia and Harman, 2011) measures test-suite strength. Release integrity is informed by SLSA (Open Source Security Foundation, 2023), Sigstore (Sigstore Project, 2021), and CycloneDX (CycloneDX Project, 2023). FIPS 197 (National Institute of Standards and Technology, 2001) defines AES, RFC 8439 (Nir and Langley, 2018) defines ChaCha20 and Poly1305, and Bernstein (2008) introduced ChaCha. NIST SP 800-90A (Barker and Kelsey, 2015) and FIPS 140-3 (National Institute of Standards and Technology, 2019) provide context for entropy and module security, although SiCaS does not claim FIPS certification.

SiCaS differs from a pure library or a pure evaluation tool by combining a masked reference integration with an assessment harness and an evidence system in one workspace. It is intentionally narrower than a general-purpose cryptographic library and intentionally more rigorous than a benchmark suite.

14 Conclusion

SiCaS is a local-first, evidence-first engineering package for first-order side-channel attenuation and leakage assessment. It provides safe-Rust masked primitives, a masked ChaCha20 reference integration, a zero-dependency assessment harness, and a reproducible evidence bundle. The current label is SAL-2 locally. The 10× performance hard ceiling is met; the 5× target is open.

The package is ready for independent review, not for production use. The next meaningful step is external verification: audit, red team, hardware leakage measurement, and production signing.

A Glossary

CPA Correlation power analysis.

DPA Differential power analysis.

EM Electromagnetic.

LTO Link-time optimisation.

MSRV Minimum supported Rust version.

SAL SiCaS assurance level.

SBOM Software bill of materials.

TVLA Test Vector Leakage Assessment.

B Claim-to-evidence matrix

Table 5: Claim-to-evidence map.

Claim	Evidence	Status
C1	Compile-fail doctests for `Secret`.	Pass
C2	Dudect and 69-configuration assembly matrix.	Pass on x86_64
C3	Synthetic TVLA/CPA/DPA, leakage model, bitsliced differential tests.	Local pass; hardware pending
C4	Entropy failure tests; Unix adapter; unsupported platforms fail closed.	Pass
C5	`cargo metadata` zero-dependency check.	Pass
C6	`forbid(unsafe_code)` in core, reference, platform, harness.	Pass
C7	Two-build hash comparison.	Pass
C8	30 Kani harnesses.	Pass
C9	Independent verification of every SAL-4 criterion.	Pending
C10	Manifest, SBOM, provenance, signing dry run.	Local pass; production pending

C Reproducibility commands

```
cargo build --workspace --release --locked
cargo test --workspace --all-targets --all-features --locked
cargo test --workspace --doc --locked
./tools/local_proof.sh quick
./tools/local_proof.sh full
```

```
cd evidence/bundle
sha256sum -c MANIFEST.sha256

python3 tools/validate_release_manifest.py \
  --manifest evidence/release/2026-10-31-sbox-dpa --skip-commit

cd evidence/release/2026-10-31-sbox-dpa
sha256sum -c SHA256SUMS
```

D Document history

Table 6: White paper revision history.

Version	Date	Code / evidence	Changes
0.1.0	2026-10-30	initial tree	Problem statement, threat model, architecture outline.
0.9.0	2026-10-31	`8c41e4b`	Full local proof, evaluation chapter, evidence summary.
1.0.0	2026-10-31	`ef13675`, `final-audit-2026-10-31-dpa`	AES S-box DPA, GPL-3.0-only, audit-ready package.

References

Elaine Barker and John Kelsey. Recommendation for random number generation using deterministic random bit generators. Technical Report NIST SP 800-90A Rev. 1, National Institute of Standards and Technology, 2015.

Daniel J. Bernstein. ChaCha, a variant of Salsa20. In *Workshop Record of the State of the Art in Stream Ciphers (SASC 2008)*, 2008.

Eric Brier, Christophe Clavier, and Francis Olivier. Correlation power analysis with a leakage model. In *Cryptographic Hardware and Embedded Systems – CHES 2004*, volume 3156 of *Lecture Notes in Computer Science*, pages 16–29. Springer, 2004. doi: 10.1007/978-3-540-28632-5_2.

Suresh Chari, Josyula R. Rao, and Pankaj Rohatgi. Template attacks. In *Cryptographic Hardware and Embedded Systems – CHES 2002*, volume 2523 of *Lecture Notes in Computer Science*, pages 13–28. Springer, 2002. doi: 10.1007/3-540-36400-5_3.

CycloneDX Project. Cyclonedx software bill of materials specification 1.5. https://cyclonedx.org/specification/overview/, 2023. Accessed 2026-10-31.

Gilbert Goodwill, Benjamin Jun, Josh Jaffe, and Pankaj Rohatgi. A testing methodology for side-channel resistance validation. In *NIST Non-Invasive Attack Testing Workshop*, 2011.

Yuval Ishai, Amit Sahai, and David Wagner. Private circuits: Securing hardware against probing attacks. In *Advances in Cryptology – CRYPTO 2003*, volume 2729 of *Lecture Notes in Computer Science*, pages 463–481. Springer, 2003. doi: 10.1007/978-3-540-45146-4_27.

ISO/IEC. Information technology – security techniques – testing methods for the mitigation of non-invasive attack classes against cryptographic modules. Technical Report ISO/IEC 17825:2016, International Organization for Standardization, 2016.

Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. doi: 10.1109/TSE.2010.62.

Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. RustBelt: Securing the foundations of the Rust programming language. In *Proceedings of the 45th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL 2018)*, pages 66:1–66:34, 2018. doi: 10.1145/3158154.

Kani Rust Verifier Project. Kani rust verifier. https://github.com/model-checking/kani, 2024. Accessed 2026-10-31.

Paul C. Kocher. Timing attacks on implementations of Diffie–Hellman, RSA, DSS, and other systems. In *Advances in Cryptology – CRYPTO '96*, volume 1109 of *Lecture Notes in Computer Science*, pages 104–113. Springer, 1996. doi: 10.1007/3-540-68697-5_9.

Paul C. Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In *Advances in Cryptology – CRYPTO '99*, volume 1666 of *Lecture Notes in Computer Science*, pages 388–397. Springer, 1999. doi: 10.1007/3-540-48405-1_25.

LLVM Project. libFuzzer: A library for coverage-guided fuzz testing. https://llvm.org/docs/LibFuzzer.html, 2024. Accessed 2026-10-31.

Stefan Mangard, Elisabeth Oswald, and Thomas Popp. *Power Analysis Attacks: Revealing the Secrets of Smart Cards*. Springer, 2007. doi: 10.1007/978-0-387-38162-6.

National Institute of Standards and Technology. Advanced encryption standard (AES). Technical Report FIPS PUB 197, U.S. Department of Commerce, 2001.

National Institute of Standards and Technology. Security requirements for cryptographic modules. Technical Report FIPS PUB 140-3, U.S. Department of Commerce, 2019.

Nicholas Nethercote and Julian Seward. Valgrind: A framework for heavyweight dynamic binary instrumentation. In *Proceedings of the ACM SIGPLAN 2007 Conference on Programming Language Design and Implementation (PLDI 2007)*, pages 89–100, 2007. doi: 10.1145/1250734.1250746.

Svetla Nikova, Christian Rechberger, and Vincent Rijmen. Threshold implementations against side-channel attacks and glitches. In *Information and Communications Security – ICICS 2006*, volume 4307 of *Lecture Notes in Computer Science*, pages 529–545. Springer, 2006. doi: 10.1007/11935308_38.

Yoav Nir and Adam Langley. ChaCha20 and Poly1305 for IETF protocols. RFC 8439, Internet Engineering Task Force, 2018.

Open Source Security Foundation. Supply-chain levels for software artifacts (SLSA) v1.0. https://slsa.dev/spec/v1.0/, 2023. Accessed 2026-10-31.

Oscar Reparaz, Josep Balasch, and Ingrid Verbauwhede. Dude, is my code constant time? In *Design, Automation & Test in Europe Conference (DATE 2017)*, pages 1697–1702, 2017. doi: 10.23919/DATE.2017.7927267.

Matthieu Rivain and Emmanuel Prouff. Provably secure higher-order masking of AES. In *Cryptographic Hardware and Embedded Systems – CHES 2010*, volume 6225 of *Lecture Notes in Computer Science*, pages 413–427. Springer, 2010. doi: 10.1007/978-3-642-15031-9_28.

Rust Project. Miri: An interpreter for Rust's mid-level intermediate representation. https://github.com/rust-lang/miri, 2024. Accessed 2026-10-31.

Tobias Schneider and Amir Moradi. Leakage assessment methodology: A clear roadmap for side-channel evaluations. In *Cryptographic Hardware and Embedded Systems – CHES 2015*, volume 9293 of *Lecture Notes in Computer Science*, pages 495–513. Springer, 2015. doi: 10.1007/978-3-662-48324-4_25.

Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitry Vyukov. AddressSanitizer: A fast address sanity checker. In *Proceedings of the 2012 USENIX Annual Technical Conference (USENIX ATC 2012)*, pages 309–318, 2012.

Sigstore Project. Sigstore: A new standard for signing, verifying, and protecting software. https://www.sigstore.dev/, 2021. Accessed 2026-10-31.